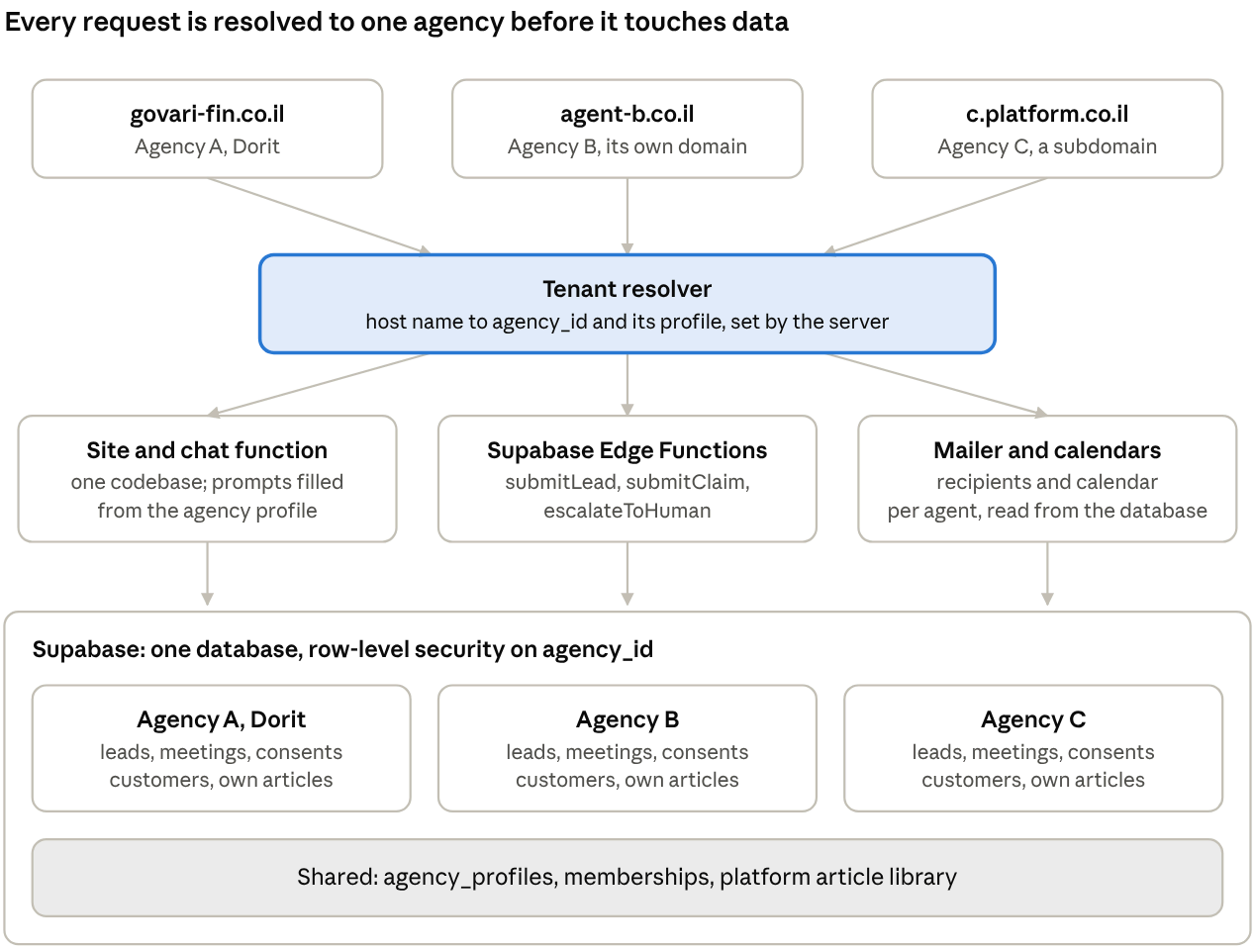
# Multi-agent platform design

Oct 10, 2026 · @Amiel Peled

Run every agent and their customers on one codebase and one Supabase backend, with Base44 removed: each row carries an `agency_id`, the database enforces who sees it, and everything now hard-coded for Dorit becomes a per-agent profile.

## At a glance




platform at a glance · 3 agencies, 1 codebase, 1 database

The host name picks the agency once, at the edge; every service below reads that agency's profile, and the database refuses rows from any other.

# Leaving Base44

Decision: nothing runs on Base44. The site is served by Vercel, the backend is Supabase — database, auth, file storage and Edge Functions — and mail stays on the existing Resend mailer. Most of the road is already built: the Supabase schema mirrors every entity, Supabase sign-in works behind `VITE_AUTH_PROVIDER=supabase`, and Vercel already builds the prerendered site and only proxies `/api` to Base44.

| Base44 piece | Used for today | Replacement |
|---|---|---|
| Hosting (`safe-arch-plan.base44.app`) | The live site | Vercel, which already builds it (`vercel.json`); the `/api` rewrite to Base44 is removed |
| 9 backend functions | Leads, claims, hand-off, meetings, content admin, personal area | Supabase Edge Functions. Both run Deno, so `Deno.env.get`, `fetch` and `Response` carry over; `createClientFromRequest` becomes the Supabase client |
| Entities `Lead`, `Contact`, `BlogPost`, `Testimonial`, `User` | Records | The Supabase tables that already mirror them; one export, checked with `scripts/reconcile-stores.mjs`, then the mirror code is deleted |
| `base44.auth` | Sign-in, OTP, password reset, Google | Supabase Auth — already wired; it becomes the only provider |
| `Core.UploadFile` | Claim documents | Supabase Storage: a private bucket, a folder per agency, signed links that expire |
| Mail through Core | Part of the notifications | The `dorit-mailer` Resend service only, which already carries most mail |
| Connectors (Outlook, Google Calendar, Docs, Sheets) | Calendar holds, the meeting doc, the event sheet | OAuth per agent, tokens encrypted in Supabase Vault — the largest piece of new work |
| 4 chat agents | Interview, support, procedures, blog | The `chat` function, below |

Two side effects worth having: the seven auth pages Base44 kept restoring (A-52 in the known-issues list) are deleted for good, and every "duplicated helper" in the functions can become one shared module, because Supabase Edge Functions can import shared code.

## Data model

Add three tables and put a non-null `agency_id` on every existing one. Supabase is the only store, because its row-level security enforces isolation in the database rather than in each function; the Base44 entities, the mirror between the two stores and the reconcile jobs all go.

The unit of tenancy is the **agency** (an independent agent is an agency of one). Inside it are **agents** — each a separate licensee — and the **customers** they serve.

| Table | New or existing | Tenant key | What it holds |
|---|---|---|---|
| `agencies` | New | — (is the tenant) | Name, status, plan, data-processing agreement date |
| `agency_profiles` | New | `agency_id` | Everything hard-coded for Dorit today: display name, phone, WhatsApp, email, notify addresses, licence, disclosure text, consent version, brand, GA4 id, domain |
| `memberships` | New | `agency_id` | `user_id` + role (owner, agent, staff) — a person can belong to more than one agency |
| `agents` | New | `agency_id` | One row per licensee: name, licence number and type, institutional affiliations, calendar connection, photo |
| `profiles` | Existing | — | Becomes identity only; the `role` column moves to `memberships` |
| `leads` | Existing | `agency_id`, `agent_id` | Enquiries, plus `referral` from the attribution work |

| Table | New or existing | Tenant key | What it holds |
|---|---|---|---|
| `meetings` | Existing | `agency_id`, `agent_id` | Bookings |
| `contacts` | Existing | `agency_id` | Chat and support contacts |
| `blog_posts` | Existing | `agency_id`, nullable | Null = the shared platform library; set = the agency's own article |
| `testimonials` | Existing | `agency_id`, `agent_id` | Verified reviews only, per agent |
| `consents` | New | `agency_id` | Which notice version a customer accepted, for which agency, when |

Today's equivalents: the Base44 entities `Lead`, `Contact`, `BlogPost`, `Testimonial`, `User`, and the Supabase tables in `supabase/migrations/20260921000000_initial_schema.sql` and `20260922100000_meetings.sql`.

A customer is not a row of their own per agency. They sign in once (`auth.users`), and `enquiries_for` — already the only door to a visitor's data — gains an `agency_id` argument, so the personal area shows each agency's enquiries separately.

## Who sees what

Access is decided by one database function, `can_see(agency_id, agent_id)`, which replaces today's `is_admin()` in every policy. A function that forgets to filter still returns nothing from another agency.

| Role | Leads and meetings | Customers' personal data | Content and profile | Other agencies |
|---|---|---|---|---|
| Platform admin | Only through an audited support grant, time-limited | Same | Shared article library | Same |
| Agency owner | All in the agency | All in the agency | Edit agency profile and articles | None |

| Role | Leads and meetings | Customers' personal data | Content and profile | Other agencies |
|---|---|---|---|---|
| Agent | Assigned to them, plus unassigned | Their own customers | Their own profile and drafts | None |
| Office staff | All in the agency, no delete | Contact fields only | None | None |
| Customer | Their own enquiries, through `enquiries_for` | Their own | None | None |
| Anonymous visitor | Insert only, agency set by the server | None | Published articles | None |

Three rules hold the line:

1. **The server sets `agency_id`, never the browser.** Backend functions resolve the agency from the request's host name. A body field naming another agency is ignored, so a crafted request cannot plant a lead in someone else's inbox.
2. **The service-role key is used inside one agency at a time.** Functions that need it (the Supabase mirror, reconcile jobs) take the resolved `agency_id` as a mandatory argument and filter on it.
3. **Platform staff do not read customer data by default.** Support access is a grant the agency owner approves, with an expiry and an audit row.

## Routing: how a visitor reaches the right agent

The host name decides the agency: `dorit.example.co.il` or the agent's own domain maps to one row in `agency_profiles`, and everything below reads from that row instead of a constant.

| Concern | Today (hard-coded for Dorit) | Per agency |
|---|---|---|
| Contact details | `src/config/contact.js` | `agency_profiles` phone, WhatsApp, email — the click events already tag `location`, so they carry over |

| Concern | Today (hard-coded for Dorit) | Per agency |
|---|---|---|
| Licence and disclosures | `LICENCE` and `CONSENT_VERSION` in `src/config/compliance.ts`, repeated in `index.html` JSON-LD, `llms.txt`, every article, `public/poa.html` | Profile fields interpolated everywhere; the consent version is per agency, because the notice names the agent |
| Chat agents | Four `base44/agents/*.jsonc` with Dorit's name inside the prompts | Decided: a backend `chat` function calls the model directly, with one prompt template per role filled from the agency's profile (see Chat below) |
| Lead mail | `NOTIFY_EMAILS` copied in each function, recipients in the `dorit-mailer` environment | `agency_profiles.notify_emails`; the mailer takes recipients per call, and sends from a verified subdomain per agency |
| Calendar | `CALENDARS` and `CALENDAR_ATTENDEES`, through Base44 connectors owned by one account | A calendar connection per agent (Microsoft Graph or Google OAuth, token stored per agent), or a booking link per agent as the first step |
| Analytics | One GA4 property, `G-LLSYPMGV58` | One platform property with `agency_id` as an event parameter, plus an optional property of the agency's own |
| Assignment | Every lead goes to Dorit | Round-robin, by topic, or to the agent whose page the visitor came from (`/a/<agent>`) |

The consequence for the code: every constant listed in `AGENTS.md` as a duplicated helper that names a person (`NOTIFY_EMAILS`, `CALENDAR_ATTENDEES`) is replaced by a lookup, which also removes the drift those duplicates were guarding against.

## Chat as a backend function

Decision: the four Base44 agents are replaced by one Supabase Edge Function, `chat`, that builds each conversation's prompt from the agency it belongs to and calls the model itself. Base44 agents are defined once per app, so they cannot speak as a different agent per domain; a function can.

One request, in order:

1. **Resolve the agency** from the host name, as every other function does. The browser sends only the role (`interview`, `support`, `procedures`, `blog`), a conversation id and the visitor's message.
2. **Check consent** against `consents` for this agency and the current notice version; no consent, no model call.
3. **Redact** the message with the existing `redact()` before it leaves the function, so an ID number typed by mistake never reaches the model provider.
4. **Build the prompt** in three layers, in this order: the platform's compliance rules from `base44/agents/COMPLIANCE.md` (fixed, an agency cannot edit them), the role template (today's prompt with Dorit's name replaced by placeholders), and the agency profile (name, licence, services, tone).
5. **Call the model** with the role's tools. The function runs the tools itself — `submitLead`, `escalateToHuman`, the article search — server-side and inside the same agency.
6. **Store and return** the reply. Store messages in `conversations` and `messages`, both keyed by `agency_id`, kept for a period the agency sets.

What this fixes beyond multi-agency: Base44 does not execute an agent's tool calls in an anonymous conversation, which is why the interview today ends in a fenced `lead` block that the page submits (A-59 in the known-issues list). With the function running the tools, that workaround and its special path in `AgentChat` go away.

What it costs:

- **A model provider contract.** The function needs its own API key and a data-processing agreement with the provider, which the meeting-prep design already listed as a blocker for document uploads.
- **Usage per agency.** Each call records tokens against `agency_id`, so cost is visible per agency and can be capped or billed.
- **Abuse limits.** A rate limit per visitor and per agency, and a maximum conversation length, enforced in the function rather than trusted to the page.
- **Testing moves with it.** The agent contract tests that pin the `.jsonc` definitions become tests of the template, the three-layer order and the tool calls; the evaluation suite in `tests/eval` runs against the function.

## Choosing the model

The chat runs on a small, cheap model, chosen by test rather than by price: on any of the three below a typical interview costs 1–2 cents, so Hebrew quality and keeping to the compliance rules decide. The `chat` function talks to the model through one adapter, so the provider is a configuration value, not code.

| Model | Input, $ per 1M tokens | Output, $ per 1M tokens | Per 1,000 interviews | Note |
|---|---|---|---|---|
| Claude Haiku 5.5 | 0.10 | 0.50 | ~$7 | Prompts under 100K tokens; the fixed prompt layers cache at $0.01 |
| GPT-5.6 Luna | 0.20 | 1.20 | ~$14 | OpenAI's everyday tier |
| Gemini 3.1 Flash-Lite | 0.25 | 1.50 | ~$18 | Paid tier only: Google trains on free-tier data; Flash prices rise 1 Jan 2027 |
| Claude Sonnet 5.5, for reference | 2.00 | 10.00 | ~$140 | Only if no small model passes |

Prices as published on 10 Oct 2026. An interview is estimated at 10 turns, about 60,000 tokens read and 2,000 written. The author of this estimate is a Claude model, so the test below, not this table, makes the choice.

The test, before phase 3 ships:

1. Run `tests/eval` and 30–50 anonymised real Hebrew interviews through each of the three small models.
2. Score each on four things: Hebrew a customer would accept as natural; no product recommendation, ever; hand-off to the licensee when the rules say so; and a complete lead at the end.
3. Take the cheapest model that passes all four, and re-run the same test whenever the provider changes model version.

Whichever wins, its business terms must say API data is not used for training, and a lawyer reviews the data-processing agreement for three points: that it covers each agency as the data owner, how long the provider keeps inputs, and the transfer of data outside Israel —

none of the three offers an Israeli region.

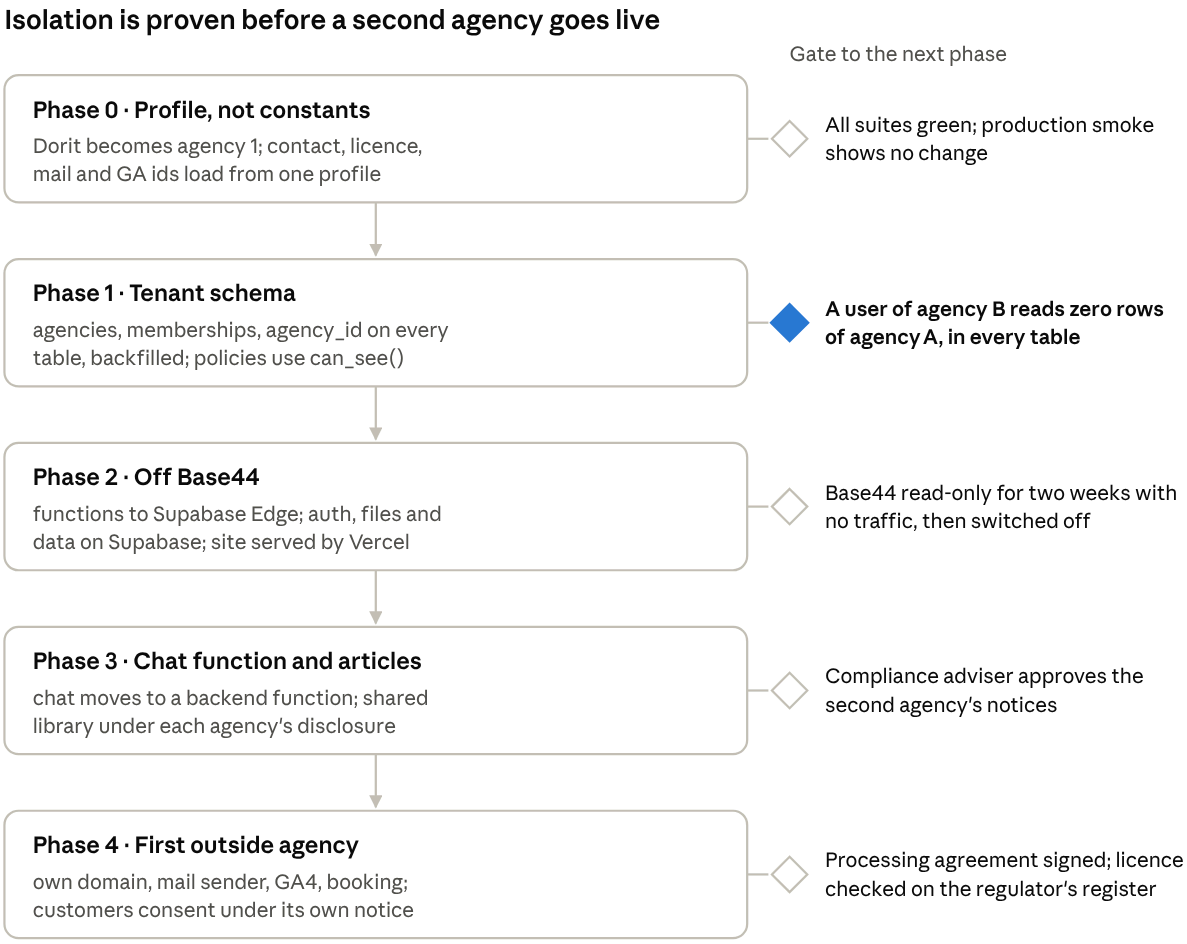
## Regulation and privacy per agent

Each agency owns its customers′ data and each agent carries their own licence; the platform is a processor serving many of them. That shapes four requirements:

- **Data controller.** The agency is the owner of the database under the Privacy Protection Law and Amendment 13; the platform signs a data-processing agreement with each agency and holds no right to use one agency′s data for another. Store the agreement date on `agencies`, and block go-live without it.
- **Licence and disclosures.** Every page, chat and article shows the licence number, type and institutional affiliations of the agent behind it — never a platform-wide statement. Verify the licence against the Capital Market Authority′s public register at onboarding and again yearly.
- **Consent is per agency.** A customer who accepted Dorit′s notice has not accepted another agent′s. `consents` stores agency, notice version and time; the chat′s consent gate reads the version from the profile.
- **The bots stay within the line.** The rules in `base44/agents/COMPLIANCE.md` (no product recommendations, hand-off to the licensee) apply to every agency unchanged; agencies can change tone and services, not those rules.

Two things that look shareable but are not: testimonials and success stories belong to the agent who earned them, and a shared article is published under each agency′s own disclosure.

# Migration




migration roadmap · 5 phases, a gate after each

Dorit's site keeps working at every step: phase 0 changes where values come from, not what they are, and no outside agency's data enters the database until phase 1's isolation tests pass. The existing contract suites carry over — the new ones are a check that every table has a non-null `agency_id` with a policy on it, and an integration test that reads across agencies and must get nothing.

# Open questions

- [ ] Chat: how long should each agency keep transcripts, and who runs the model test and the lawyer's review of the winner's data-processing agreement?
- [ ] Is an "agency" always a legal business, or can two independent agents share one agency and still keep separate customer lists?

- [ ] Does a customer who moves from one agent to another inside an agency need new consent?
- [ ] Calendar: build per-agent OAuth now, or start with a booking link per agent?
- [ ] Pricing and billing per agency — out of scope here, but it decides whether `agencies.plan` is needed in phase 1.
- [ ] Who checks each new agency's disclosures before go-live — Dorit's compliance adviser, or one the platform retains?